MASTER SISE / ATELIER DATA

Apache Hop & les ETL

De la donnée brute au flux maîtrisé

Présentation personnalisée • 2026

Fil conducteur

01 COMPRENDRE

Le besoin
Transformer des données dispersées en information exploitable.

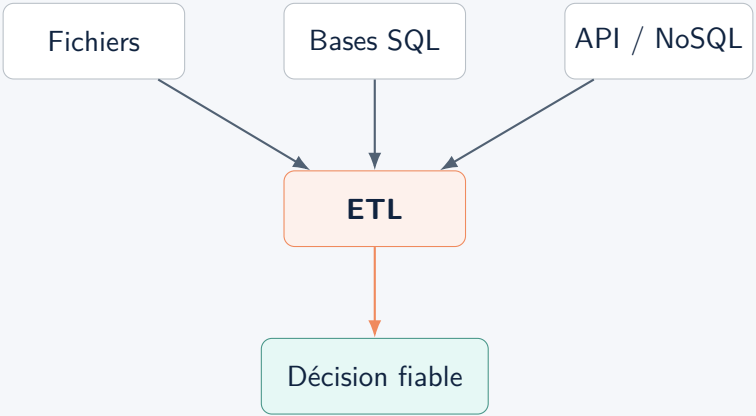
L'outil
Découvrir Apache Hop comme plateforme visuelle d'orchestration.

La pratique
Construire un pipeline, le relier et l'exécuter.

Une même idée : **décrire le flux avant d'écrire du code**.

Pourquoi les ETL ?

02 LE PROBLÈME



Extraire → transformer → charger

ETL en trois gestes

03 LE MODÈLE

E Extract
Lire les sources : fichiers, bases relationnelles, services et flux.

T Transform
Nettoyer, filtrer, joindre, enrichir et contrôler la qualité.

L Load
Déposer la donnée prête à l'emploi dans une cible.

À retenir

Un flux ETL rend les traitements **visibles**, **répétables** et **observables**.

Un marché riche en solutions

04 ÉCOSYSTÈME

Open source

Talend • Pentaho • Apache NiFi • Apache Hop

Data engineering

StreamSets • Airflow • Spark

Solutions historiques

Informatica • DataStage

Suites éditeurs

Oracle Data Integrator • SQL Server Integration Services

Apache Hop, en une phrase

05 DÉFINITION

Une plateforme visuelle pour orchestrer données, métadonnées et traitements.



Les points forts de Hop

06 PROMESSE

Visuel

Une approche centrée sur le résultat plutôt que sur la complexité du code.

Extensible

Un écosystème de plugins pour adapter la plateforme.

Productif

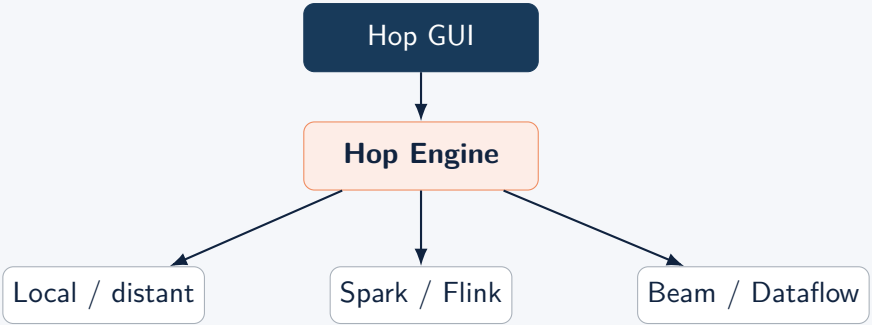
Des transformations réutilisables et une conception rapide.

Portable

Du poste local au cloud, aux conteneurs et à Kubernetes.

Architecture : un moteur, plusieurs contextes

07 ARCHITECTURE



Environ 400 plugins : lire, écrire, combiner, enrichir et nettoyer.

Des runtimes flexibles

08 EXÉCUTION

- **Hop GUI** : concevoir les pipelines et les workflows dans un environnement visuel.
- **Hop Engine** : exécuter localement ou à distance.
- **Moteurs distribués** : s'appuyer sur Spark, Flink ou Beam selon le contexte.
- **Modes de traitement** : batch, continu ou hybride.

local cloud conteneurs Kubernetes

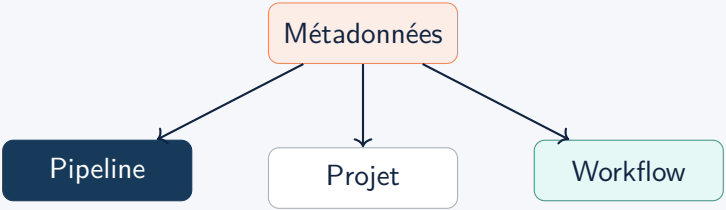
Cas d'utilisation

09 TERRAIN

- Grandes volumétries et chargements parallèles
- Entrepôts de données et dimensions à évolution lente
- Capture des changements et clés de substitution
- Intégration SQL, fichiers et bases NoSQL
- Migration entre applications et bases
- Profilage et nettoyage de données

Le vocabulaire Hop

10 CONCEPTS



Pipeline ou Workflow ?

11 DISTINCTION

Pipeline

Travail concret sur les données. Les transformations s'exécutent en parallèle et font circuler des lignes.

Exemple : lire un CSV, calculer une valeur, écrire dans une base.

Workflow

Orchestration des actions. Les étapes s'enchaînent selon un résultat vrai ou faux.

Exemple : lancer un pipeline, vérifier son statut, archiver le résultat.

Action, Hop, Transformation

12 BRIQUES

Action	Opération d'un workflow ; renvoie un statut qui influence la suite.
Hop	Lien entre deux actions ou deux transformations ; porte un statut ou des données.
Transformation	Unité de travail d'un pipeline : lecture, jointure, enrichissement ou nettoyage.

Un pipeline comme graphe acyclique

13 MODÈLE MENTAL



DAG : Directed Acyclic Graph, donc pas de boucle dans le flux.

Projets et environnements

14 INDUSTRIALISER

Projet

Regroupe configurations, variables, métadonnées, pipelines et workflows. Il peut hériter d'un projet parent.

Environnement

Définit la configuration réelle d'exécution : connexions, chemins, paramètres et secrets.

dev → uat → prd

Exemple d'environnement

15 CONFIGURATION

Projet : Sales
Environnement : dev
Hôte : 10.0.0.1
Base : clients

Même logique métier, paramètres adaptés à chaque contexte.

Installer Apache Hop

16 PREMIERS PAS

1. Télécharger une distribution depuis la page officielle du projet.
2. Installer un JDK compatible, notamment Java 11 selon la version utilisée.
3. Décompresser Hop dans le dossier de son choix.
4. Lancer Hop GUI avec le script fourni par la distribution.

Conseil

Commencer par un environnement de test isolé avant de connecter des données sensibles.

Construire un pipeline : la méthode

17 ATELIER

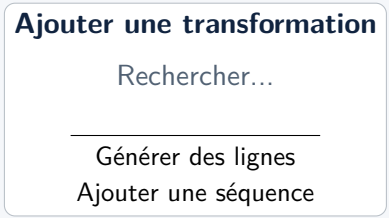
1. Créer un nouveau pipeline depuis la barre d'outils ou **Fichier → Nouveau → Pipeline**.
2. Ajouter les transformations nécessaires dans le canevas.
3. Relier les transformations avec des Hops.
4. Paramétrer, sauvegarder puis exécuter.

Concevoir → Relier → Tester → Observer

Ajouter des transformations

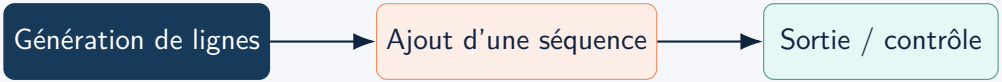
18 CONCEPTION VISUELLE

- Cliquer sur le canevas du pipeline.
- Rechercher une transformation dans la boîte de dialogue.
- Valider avec la souris ou la touche Entrée.
- Répéter pour chaque étape du flux.



Exemple : générer et numéroter

19 TRANSFORMATION



Un petit flux suffit pour valider la logique avant de brancher une source réelle.

Créer un Hop

20 CONNEXION

Shift-drag
Maintenir Shift, cliquer sur la première transformation et glisser vers la seconde.

Scroll-drag
Utiliser le bouton central de la souris pour relier deux éléments.

Menu contextuel
Ouvrir la boîte de dialogue et choisir *Créer un saut*.

Exécuter et observer

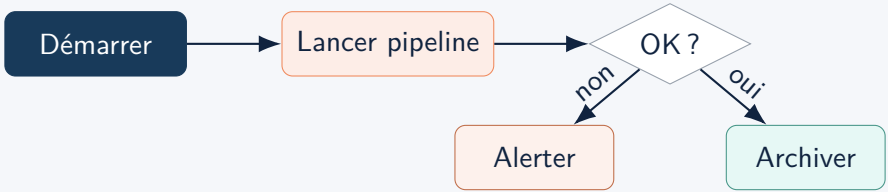
21 VALIDATION



- Vérifier le nombre de lignes entrantes et sortantes.
- Contrôler les erreurs de connexion et de typage.
- Tester sur un petit volume avant le traitement complet.

Orchestrer avec un workflow

22 WORKFLOW



Sauvegarder pour rendre durable

23 BONNES PRATIQUES

- Nommer clairement pipelines, workflows et variables.
- Séparer développement, recette et production.
- Centraliser les paramètres d'environnement.
- Versionner les artefacts Hop.
- Documenter les sources et les sorties.
- Prévoir logs, contrôles et reprise sur erreur.

Synthèse

24 À RETENIR



- Les ETL structurent le passage de la donnée brute à la donnée utile.
- Apache Hop sépare clairement traitement, orchestration et configuration.
- La conception visuelle accélère l'expérimentation sans sacrifier l'industrialisation.

Merci

Des questions sur vos flux de données ?

Apache Hop • ETL • Data engineering